Project #1 Report
Constrained-Random Verification Of The FIFO
EEE 4701 / EEE 5703 · Verification & Validation Of Digital Systems · Fall 2026

Team	Group #15
Members	Minh Hoang Ho | Matias Guillen | William Wright | Adrian Emeterio
Repo, & The Commit Or Tag Graded	GitHub Repository
Demo Video Link	Demo Link
Simulator & Versions	Icarus Verilog 12.0 | cocotb 1.9.2 | pyvsc 0.9.5

1. Summary

Measure	Value	Where The Evidence Is
Released Pool: Found / Total	8 / 8	FINDINGS.md
Coverage: Reached / Target	100% / 90%	The coverage_report.txt file generated
Tests: Directed / Constrained-Random	9 / 2	test_fifo.py | seeded_regression.py | transactions.py
Seeds In The Regression	15 pyvsc Seeds & 5 Plain Random Seeds	Section 7
Survivors Analyzed	2 Fault Classes Since No Released Mutant Survived	Section 6 | FINDINGS.md

What We Verified, How, & What Is Left:

We checked the 8 by 8 FIFO against every rule in the spec which covers reset, writes, reads, writing while full, reading while empty, reading and writing at the same time, the flags, the order of the data, and the pointers wrapping around. Our testbench uses 9 directed tests, a plain random test with 5 seeds, and a pyvsc random test with 15 seeds. On every cycle, the scoreboard compares the FIFO's count, full, empty, and output data to a reference queue. All 11 tests pass, all 8 released mutants were caught, and all 37 coverage bins were hit, but we do not check rd_data while the FIFO is empty, other WIDTH and DEPTH values, anything before the first reset, or the almost_full flag.

2. Plan & Traceability

Spec Requirement	Feature ID	Test(s)	Coverpoint Or Bin	Status
Reset	F1 | F12	test_reset | test_reset_in_use	C3 | Write During Reset	Covered
Write	F2	test_fill_drain | random tests	C2 Write | X1	Covered
Write While Full	F3	test_overflow | random tests	Write While Full	Covered
Read	F4	test_fill_drain | random tests	C2 Read | X1	Covered
Read While Empty	F5	test_underflow | random tests	Read While Empty	Covered
rd_data Shows The Head	F6	Scoreboard | test_all_bits	Scoreboard Check	Partly
Simultaneous Read & Write	F7	test_read_write | random tests	Both At Empty & Full	Covered
Flags	F8	Scoreboard | test_overflow | test_underflow	C1 empty and full	Covered
Ordering	F9	Scoreboard | test_wrap | random tests	Scoreboard Check	Covered
Corner Cases, Pointer Wrap	F10	test_wrap | random tests	Write Wrap | Read Wrap	Covered
Parameters, WIDTH	F11	test_all_bits	None	Partly
Parameters, DEPTH	None	None	None	Not Covered

Requirements With No Test, & Why:

We do not test other WIDTH and DEPTH values because we only build the FIFO at 8 by 8. Our coverage bins also only work for a depth of 8. We do not test anything before the first reset because every test starts with a reset and the spec does not say what the outputs should be before that. On top of this ee do not test the almost_full flag.

3. The Environment

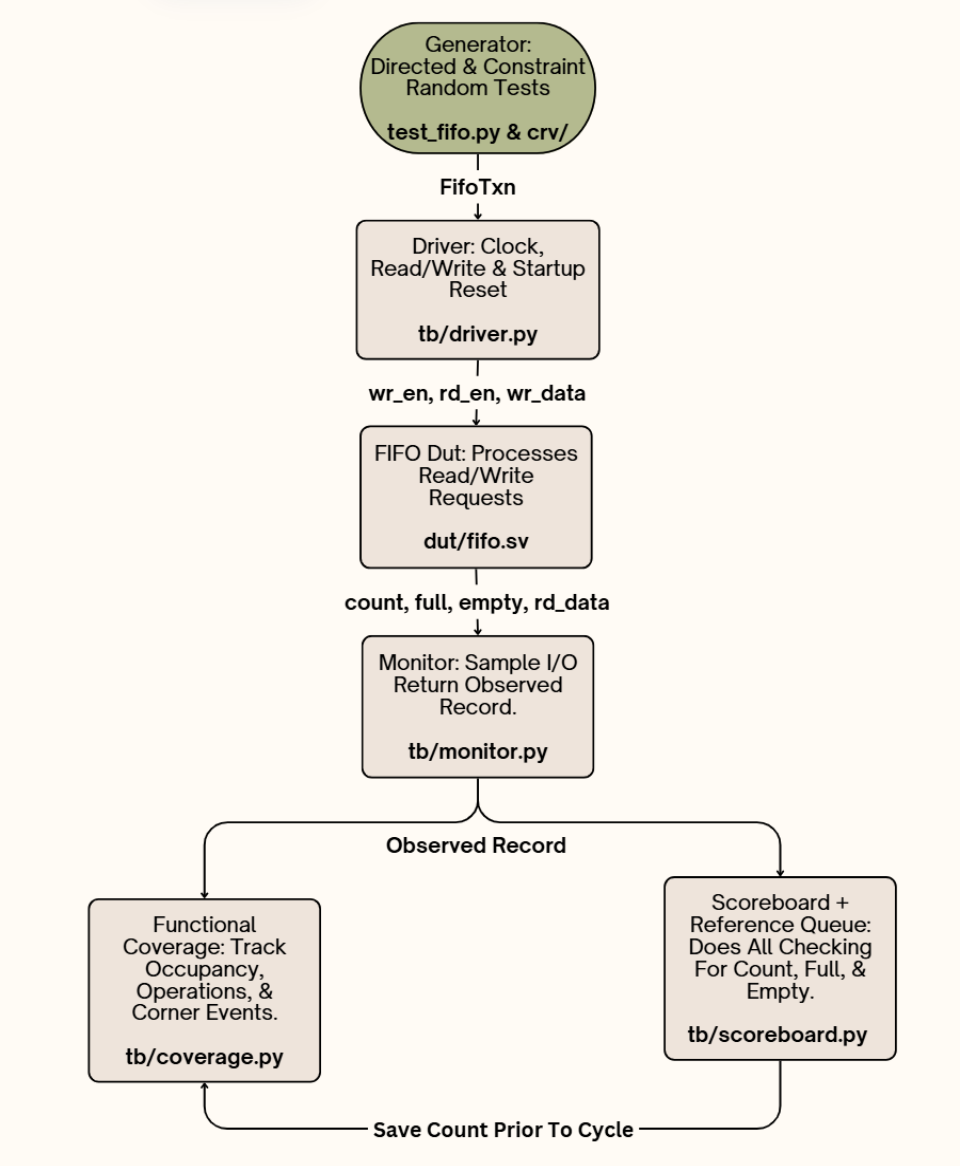


Figure #1. Our Verification Environment: Generator, Driver, FIFO, Monitor, Scoreboard With Its Reference Queue, & Coverage.

Component	File	Its One Job	Reused From (HW4, HW5, Or New)
Driver	tb/driver.py	Controls The FIFO's Clock & Reset & Applies Read/Write Requests At The Correct Clock Timing	HW4
Monitor	tb/monitor.py	Records The FIFO's Input & Output Signals After Each	HW4

Component	File	Its One Job	Reused From (HW4, HW5, Or New)
		Clock Cycle & Returns Them For The Scoreboard To Check	
Scoreboard (Reference Queue)	tb/scoreboard.py	Keeps A Reference Queue Of What The FIFO Should Hold & Checks The FIFO's Count, Flags, & Output Against It Every Cycle	HW4
Transaction & Constraints	crv/transactions.py	Sets The Bounds Of Legal Stimulus That Can Be Applied To The FIFO	HW5
Coverage Collector	tb/coverage.py	Tracks Which Of The 37 Coverage Bins Were Hit & Writes The Coverage Report	New

What The Scoreboard Checks, & On Which Cycles:

After every clock cycle, the monitor reads the FIFO's outputs and the scoreboard updates its reference queue with that cycle's wr_en, wr_data, and rd_en. The scoreboard then checks count, full, and empty on every cycle, and checks rd_data against the first item in the queue whenever the FIFO is not empty. During reset the scoreboard is skipped, so the test itself checks that count is 0, empty is 1, and full is 0 on every reset cycle, and then clears the queue.

4. Constrained-Random Stimulus

Our Transaction Class & Constraints (Excerpt From crv/)

```
@vsc.randobj
class FifoTxn:
    """
    FifoTxn contains the data needed for one cycle of FIFO stimulus.
    Variables are random rather than driven and constrained to fit within the legal bounds of the
FIFO spec.
    The phase variable should be updated and set before a randomize call.
    """

    def __init__( self, width = WIDTH ):
        self.phase = vsc.uint8_t( FILL )
        self.roll_w = vsc.rand_uint8_t( )
        self.roll_r = vsc.rand_uint8_t( )
        self.wr_en = vsc.rand_bit_t( 1 )
        self.rd_en = vsc.rand_bit_t( 1 )
```

```
        self.wr_data = vsc.rand_bit_t( width )

    @vsc.constraint
    def c_rolls( self ):
        self.roll_w in vsc.rangelist( vsc.rng( 0, 99 ) ) # Uniform rolls.
        self.roll_r in vsc.rangelist( vsc.rng( 0, 99 ) )
        vsc.solve_order( self.roll_w, self.wr_en ) # Solve the roll before the enable so we get the
expected probability.
        vsc.solve_order( self.roll_r, self.rd_en )

    @vsc.constraint
    def c_shape( self ):
        # FILL: write 85% of cycles, read 10% (the overlap keeps simultaneous read+write alive).
        with vsc.if_then( self.phase == FILL ):
            with vsc.implies( self.roll_w < 85 ):
                self.wr_en == 1
            with vsc.implies( self.roll_w >= 85 ):
                self.wr_en == 0
            with vsc.implies( self.roll_r < 10 ):
                self.rd_en == 1
            with vsc.implies( self.roll_r >= 10 ):
                self.rd_en == 0
        # DRAIN: the mirror image.
        with vsc.else_then( ):
            with vsc.implies( self.roll_w < 10 ):
                self.wr_en == 1
            with vsc.implies( self.roll_w >= 10 ):
                self.wr_en == 0
            with vsc.implies( self.roll_r < 85 ):
                self.rd_en == 1
            with vsc.implies( self.roll_r >= 85 ):
                self.rd_en == 0

    @vsc.constraint
    def c_data( self ):
        # A top-level dist is honored: 1 in 5 words is an all-zeros or all-ones pattern.
        vsc.dist( self.wr_data, [ vsc.weight( 0, 10 ), vsc.weight( ( 1 << WIDTH ) - 1, 10 ),
vsc.weight( vsc.rng( 1, ( 1 << WIDTH ) - 2 ), 80 ) ] )
```

How The Constraints Keep Every Transaction Legal:

The constraints in our transaction class act as a specification describing what the FIFO design is allowed to receive. By setting those bounds, the solver is inherently unable to generate illegal stimulus. The bounds are the part of CRV that cannot be random, we had to understand what the DUT could and could not do before explicitly setting them.

How They Reach The Corners:

The corner cases for the FIFO lie at its endpoints, either read while empty or write while full. We purposefully shape the distribution of our CRV to reach these corners by using a phase variable to track state and influence probability. To ensure the FIFO fills up, we need the percentage of writes to outweigh reads. This corresponds to the FILL phase state. Once full, we want the phase to flip and the FIFO to start emptying. This corresponds to the DRAIN state where reads outweigh writes. One problem that can arise is, if we flip from FILL to DRAIN immediately (or vice versa), the FIFO isn't able to spend much time at the corners and in some cases won't reach them at all. To counteract this, we instruct the phase to dwell at the corners for 3 cycles once it reaches them.

Measured Occupancy Over One Regression Run (Seed 1, 500 Cycles); One Column Per Occupancy, 0 To DEPTH

Occupancy	0	1	2	3	4	5	6	DEPTH-1	DEPTH (Full)
Cycles	79	55	42	43	46	47	40	58	90

5. Coverage Closure

The Coverage Model

Coverpoint Or Cross	Bins	The Bug Class It Is Aimed At
C1 How Full The FIFO Is Before The Cycle	Empty At 0, Near Empty At 1, Middle From 2 To 6, Near Full At 7, And Full At 8	Most FIFO Bugs Happen Right At The Edges So 0, 1, 7, And 8 Each Get Their Own Bin While 2 To 6 Act The Same And Share One
C2 What The FIFO Was Asked To Do That Cycle	To Stay Idle, Write, Read, Or Both	These Are The Only 4 Possible Options Of wr_en And rd_en
C3 Reset While An Enable Is Held High	Write Held During Reset	Reset Has To Clear The FIFO Even If A Write Is Requested At The Same Time And This Is Our Check For An Illegal Input
C4 Corner Events	Write While Full, Read While Empty, Both At Empty, Both At Full, Both In The Middle, Write Pointer Wrap, Read Pointer	These Are The Spots Where FIFO Bugs Usually Hide So Each One Gets Its Own Bin

Coverpoint Or Cross	Bins	The Bug Class It Is Aimed At
	Wrap, And Write During Reset	
X1 C1 Crossed With C2 For 20 Bins	Every C1 Bin Crossed With Every C2 Bin	Bugs That Only Show Up At The Edges Like Writing While Full To Check For Overflow, Reading While Empty To Check For Underflow, And Reading And Writing Together At Full Or Empty Where The FIFO Could Accept Something It Should Not

Illegal & Ignore bins

Bin	Illegal Or Ignore	Why (Spec Line, Or The Argument)
None	None	The Spec's Write Rule, Read Rule, And Simultaneous Read And Write Rule Cover Each Request And Even Writing While Full And Reading While Empty Should Follow The Spec

Coverage Report: The Lines That Show The Result

```
FIFO Coverage: 37/37 Bins (100.0%)
Occupancy: 5/5
Occupancy Counts: 0: 1682, 1: 1234, 2: 913, 3: 876, 4: 908, 5: 803, 6: 836, 7: 1075, 8: 1622
Operations: 4/4
Occupancy X Operation: 20/20
Corner Events: 8/8
Ignored Requests (Defined By Spec): ['read_empty', 'write_full']
Reset/Write Overlap: Hit
Missing Crosses: []
Missing Events: []
```

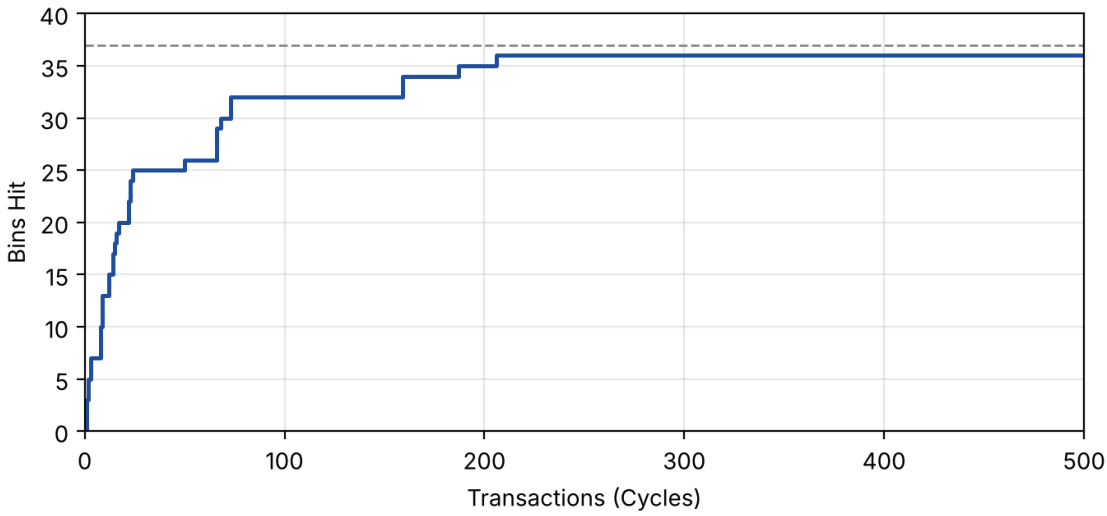


Figure #2. Coverage Versus Transactions For Seed 42

The Last Holes, & What Closed Them

Hole	Why It Was Rare	What Closed It (Constraint Change, Or A Directed Test And Why)
Both At Full, Both At Empty	Both Enables Have To Be High While The FIFO Sits At A Corner. From The Constraints, This Has An 8.5% Chance Per Cycle (85% × 10%), And Only On The Few Cycles The FIFO Is Actually At Full Or Empty	Constraint Change. Giving The Phase Class Time To Dwell At The Corner For 3 Cycles Gives The Overlap Several Chances To Occur
Write During Reset	Random Stimulus Never Drives rst_n. In The Regression, reset_during_test Holds Both Enables Low, So This Bin Has A 0% Chance Of Being Hit By CRV	Directed Test. test_reset_in_use Holds wr_en High During Reset On Purpose. Reset Is Not Part Of The Transaction Class, So No Constraint Change Could Reach This Bin Without Moving Reset Into The Randomized Fields

Why The Curve Flattened Where It Did:

The last bins were reading and writing together at full, at empty, at near empty, and at near full. These are slow because each phase only turns on the other enable 10% of the time so both enables have to land on the same cycle while the FIFO is sitting at that exact count. After cycle

pass around the 200 mark, the line stays around 36 since write during reset can not happen in a random cycle and is only hit by test_reset_in_use.

6. Mutation Results & Survivors

Released Pool, As Run

```
$ just mutate project_one
[baseline] running clean fifo.sv through project_one ...
  clean RTL: 11 tests passed.
  [found  ] full_offbyone       (test-failed)
  [found  ] empty_offbyone      (test-failed)
  [found  ] ignore_full         (test-failed)
  [found  ] ignore_empty        (test-failed)
  [found  ] count_wr_dec        (test-failed)
  [found  ] rd_data_wrptr       (test-failed)
  [found  ] mem_corrupt         (test-failed)
  [found  ] wrptr_wrap_early    (test-failed)
==== found/total = 8/8  =  100.0%  (threshold 90%) ====
```

Mutant (Released Id)	What It Changes	Reached?	Observed?	Why We Missed It	What Would Catch It
Wrong rd_data While Empty Which Was A Fault Class Outside The Released Pool	The FIFO Works Normally But Shows A Wrong Value On rd_data Whenever It Is Empty	Yes	No	Our Tests Will Empty The FIFO Many Times, But We Never Check rd_data While Empty	A Scoreboard Check For That Value
A Bug That Only Happens At Another Size Which Was A Fault Class Outside The Released Pool	The FIFO Works At 8 By 8 But Breaks At Another Size	No	Yes	We Only Build The FIFO At WIDTH = 8 & DEPTH = 8 So	Running The Same Tests Again At DEPTH = 4, DEPTH = 16, & A Different WIDTH

What We Changed, & Which Survivors It Caught:

We started with only the starter smoke test, which caught 1 of the 8 released mutants, count_wr_dec. We then added the scoreboard check on every cycle, 8 more directed tests, and

the two random tests which brought the score to 8 of 8. Those changes caught the other 7 mutants, which were full_offbyone, empty_offbyone, ignore_full, ignore_empty, rd_data_wrptr, mem_corrupt, and wrptr_wrap_early.

7. Reproducibility

The Exact Commands For Every Number In This Report

```
# From code/
$ source /opt/course/cocotb-env/bin/activate
$ just mutate project_one
$ cat tb/project_one/coverage_report.txt
```

Seed	Command	Found / Total (Released)	Coverage
All 15 pyvsc Seeds & Plain Random Seeds 1 To 5	just mutate project_one	8 / 8 (100%)	37 / 37 Bins (100%)
All 15 pyvsc Seeds & Plain Random Seeds 1 To 5	cat tb/project_one/coverage_report.txt	N/A	37 / 37 Bins (100%)

8. Illegal & Undefined Behavior

What We Added (Cover Point Or Assertion):

We added a write during reset coverpoint which is hit when rst_n is 0 while wr_en is 1. The test_reset_in_use test holds wr_en and rd_en high during reset with junk data 0x5A on wr_data so the FIFO gets a reset and a write on the same clock edge.

Was It Hit, & What Did It Show?

Yes, the coverage report shows “Reset/Write Overlap: Hit” at the very end. On every reset cycle the test checks that count is 0, empty is 1, and full is 0. After reset, the FIFO is still empty and the junk data was never stored so reset occurs over the write.

9. What This Suite Does Not Establish

1) Wrong rd_data while the FIFO is empty. This is an observability gap since we never check rd_data when the FIFO is empty. We would need to add a spec rule for what rd_data should show and a scoreboard check for it.
2) Bugs that only show up at other WIDTH or DEPTH values. This is a reachability gap since we only build the FIFO at 8 by 8. We would need to run the same tests at other sizes.
3) Anything before the first reset. This is a reachability gap since every test resets first. We would need a test that checks the outputs before reset.

If We Had Another Day:

We would run all the tests again at a couple of other sizes like DEPTH = 4 and DEPTH = 16 since that is our biggest gap and it would barely need any new code.

10. AI use

Summary For AI Log:

We used AI to update seeded_regression.py so it worked with the transactions.py file and to make a Python file that helped create the Figure #2 graph. AI was also used to explain the phase, hysteresis, and roll ideas from the course slides that our constraints are based on, and to split the scoreboard and coverage into their own files, add the write during reset coverage event, and connect them to the test file. Before using any code we checked that it matched the names of our own files. We fixed run_seed to use the scoreboard instead of the model, added more seeds to get to 15, and changed the coverage file to coverage_report.txt, and since matplotlib is not on eustis the graph was made from the printed numbers instead. We checked everything on eustis where all 11 tests passed, 8 of 8 mutants were found, and coverage was 37 of 37 bins.

11. Team

Member	Led	Reviewed
Matias Guillen	tb/driver.py | tb/monitor.py	The Markdown Files & Code
Minh Hoang Ho	test_fifo.py | crv/seeded_regression.py	The Markdown Files & Code
William Wright	crv/transactions.py	The Markdown Files & Code
Adrian Emeterio	tb/scoreboard.py | tb/coverage.py	The Markdown Files & Code